

**Ex.No:1      BASICS OF NETWORK CONFIGURATIONS FILES AND NETWORKING  
COMMANDS IN LINUX**

**Network Configuration Files in Linux:**

The important network configuration files in Linux operating systems are

**1. /etc/hosts**

This file is used to resolve hostnames on small networks with no DNS server. This text file contains a mapping of an IP address to the corresponding host name in each line. This file also contains a line specifying the IP address of the loopback device i.e, *127.0.0.1* is mapped to *localhost*.

A typical *hosts* file is as shown

```
127.0.0.1          localhost
127.0.1.1 anil-300E4Z-300E5Z-300E7Z
```

**2. /etc/resolv.conf**

This configuration file contains the IP addresses of DNS servers and the search domain.

A sample file is shown

```
# DO NOT EDIT THIS FILE BY HAND -- YOUR CHANGES WILL BE OVERWRITTEN
nameserver 127.0.1.1
```

**3. /etc/sysconfig/network**

This configuration file specifies routing and host information for all network interfaces. It contains directives that are global specific. For example if NETWORKING=yes, then /etc/init.d/network activates network devices.

**4. /etc/nsswitch.conf**

This file includes database search entries. The directive specifies which database is to be searched first.

**Networking Commands in Linux:**

The important Linux networking commands are

**1. ifconfig:**

- Assign addresses to network interfaces.
- Display the current network interface configuration.

- Examine the status of a communications interface.
- Set up hardware addresses.
- Enable or disable network interfaces.
- **The basic ifconfig syntax looks like this:**  
*ifconfig [interface\_name\_optional] [arguments\_optional]*
- [interface] - The network interface to configure or show information for. The parameter is optional, and not specifying an interface shows the status of all active interfaces.
- [options] - Command-line options to perform specific actions or configure certain parameters.
- The parameter is also optional.

## 2. ping:

- Is a network utility that tests if a host is reachable on an IP network.
- It can be used to troubleshoot connectivity, reachability, and name resolution.
- Especially useful for assessing network performance.

**The ping command has the following syntax:**

*ping [options] [hostname/IP address]*

## 3. traceroute:

- Identifies the path packets take from source to destination.
- Helps to calculate the transfer times between points.
- Helps to locate where traffic slows down between the source and destination.

**The syntax for the traceroute command is:**

*traceroute [options] [hostname/IP]*

- The [hostname/IP] parameter is required, while additional options control whether to perform DNS lookups, the TTL parameter, and the packet type.

## 4. nslookup:

- Stands for “Name Server Lookup”
- It is a useful command for getting information from the DNS server.
- It is a network administration tool for querying the Domain Name System (DNS) to obtain domain name or IP address mapping.
- It is also used to troubleshoot DNS-related problems.
- **The syntax for the nslookup command is:**

*nslookup [option] [domain]*

- [options] refer to additional parameters that modify the behavior of the command.
- [domain] denotes the domain name or IP address to be queried.

### ssh:

- SSH (Secure Shell or Secure Socket Shell) is a network protocol that gives users -- particularly systems administrators -- a secure way to access a computer over an unsecured network.
- Provides encrypted data communications between two computers connecting over an open network, such as the internet.
- SSH enables the following functions:
- Secure remote access to SSH-enabled network systems or devices for users, as well as automated processes.
- Secure and interactive file transfer sessions.
- Automated and secured file transfers.
- Secure issuance of commands on remote devices or systems.
- Secure management of network infrastructure components.
- Authentication and host-based authentication.
- Remote execution of commands.

### scp:

- SCP (Secure Copy Protocol) is a network protocol that securely copies files/folders between Linux (Unix) systems on a network.
- To transmit, use the scp command line utility, a safer variant of the cp (copy) command.
- SCP protects your data while copying across an SSH (Secure Shell) connection by encrypting the files and the passwords.
- **The syntax for using the scp command is:**

*scp [option] [user\_name@source\_host:path/to/source/file][user\_name@target\_host:target/path]*

### telnet:

- The telnet command (short for teletype network) utilizes the telnet network protocol to connect to remote machines and manage them through a TCP/IP network.
- The protocol uses port 23 by default to establish a connection.
- Although telnet is similar to SSH, the two are different because SSH is a much more secure option as it uses encryption. telnet, on the other hand, sends data without encryption, making it an easy target for hackers.
- **The syntax for the telnet command is:**

*telnet [options] [remote\_server\_address] [port]*

- The [options] and [port] parameters are optional. For [remote\_server\_address], telnet accepts symbolic and numeric addresses.

### ftp:

- FTP (File Transfer Protocol) is a network protocol used for transferring files from one computer system to another.
- **The Linux ftp command uses the following basic syntax:** *ftp [options] [IP]*

**Ex.No.2            TO FAMILIARIZE AND UNDERSTAND THE USE AND FUNCTIONING OF  
SYSTEM CALLS USED FOR OPERATING SYSTEM AND NETWORK  
PROGRAMMING IN LINUX.**

**System calls of Linux operating systems**

**1. ps**

This command tells which all processes are running on the system when *ps* runs. *ps -ef*

| UID  | PID | PPID | C | STIME | TTY     | TIME     | CMD            |
|------|-----|------|---|-------|---------|----------|----------------|
| root | 1   | 0    | 0 | 0     | 13:55 ? | 00:00:01 | /sbin/init     |
| root | 2   | 0    | 0 | 0     | 13:55 ? | 00:00:00 | [kthreadd]     |
| root | 3   | 2    | 0 | 0     | 13:55 ? | 00:00:00 | [ksoftirqd/0]  |
| root | 4   | 2    | 0 | 0     | 13:55 ? | 00:00:01 | [kworker/0:0]  |
| root | 5   | 2    | 0 | 0     | 13:55 ? | 00:00:00 | [kworker/0:0H] |
| root | 7   | 2    | 0 | 0     | 13:55 ? | 00:00:00 | [rcu_sched]    |
| root | 8   | 2    | 0 | 0     | 13:55 ? | 00:00:00 | [rcuos/0]      |

-----

This command gives processes running on the system, the owners of the processes and the names of the processes. The above result is an abridged version of the output.

**2. fork**

This system call is used to create a new process. When a process makes a *fork* system call, a new process is created which is identical to the process creating it. The process which calls *fork* is called the parent process and the process that is created is called the child process. The child and parent processes are identical, i.e, child gets a copy of the parent's data space, heap and stack, but have different physical address spaces. Both processes start execution from the line next to *fork*. Fork returns the process id of the child in the parent process and returns 0 in the child process.

```
#include<stdio.h> void main()
{
int pid;
pid = fork(); if(pid > 0)
{
printf (" Iam parent\n");
}
```

```

else
{
printf("Iam child\n");
}
}

```

The parent process prints the first statement and the child prints the next statement.

### 3. **exec**

New programs can be run using *exec* system call. When a process calls *exec*, the process is completely replaced by the new program. The new program starts executing from its main function.

A new process is not created, process id remains the same, and the current process's text, data, heap, and stack segments are replaced by the new program. *exec* has many flavours one of which is *execv*.

*execv* takes two parameters. The first is the pathname of the program that is going to be executed. The second is a pointer to an array of pointers that hold the addresses of arguments. These arguments are the command line arguments for the new program.

### 4. **wait**

When a process terminates, its parent should receive some information regarding the process like the process id, the termination status, amount of CPU time taken etc. This is possible only if the parent process waits for the termination of the child process. This waiting is done by calling the *wait* system call. When the child process is running, the parent blocks when *wait* is called. If the child terminates normally or abnormally, *wait* immediately returns with the termination status of the child. The *wait* system call takes a parameter which is a pointer to a location in which the termination status is stored.

### 5. **exit**

When *exit* function is called, the process undergoes a normal termination.

### 6. **open**

This system call is used to open a file whose pathname is given as the first parameter of the function. The second parameter gives the options that tell the way in which the file can be used.

```
open(filepathname , O_RDWR);
```

This causes the file to be read or written. The function returns the file descriptor of the file.

### 7. **read**

This system call is used to read data from an open file. `read(fd, buffer, sizeof(buffer));`

The above function reads `sizeof(buffer)` bytes into the array named `buffer`. If the end of file is encountered, 0 is returned, else the number of bytes read is returned.

### 8. **write**

Data is written to an open file using *write* function. `write(fd, buffer, sizeof(buffer));`

## **System calls for network programming in Linux**

### **Creating a socket**

**`int socket (int domain, int type, int protocol);`**

This system call creates a socket and returns a socket descriptor. The *domain* parameter specifies a communication domain; this selects the protocol family which will be used for communication. These families are defined in <sys/socket.h>. In this program the AF\_INET family is used. The *type*

parameter indicates the communication semantics. SOCK\_STREAM is used for tcp connection while SOCK\_DGRAM is used for udp connection. The *protocol* parameter specifies the protocol used and is always 0. The header files used are <sys/types.h> and <sys/socket.h>.

**Ex.No:3**

**IMPLEMENTATION OF CLIENT-SERVER COMMUNICATION USING  
SOCKET PROGRAMMING & TCP AS TRANSPORT LAYER**

**Program:**

**Client Side:**

```
#include <stdio.h>
#include <stdlib.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <string.h>
int main() {
    int socdef, readval;
    struct sockaddr_in addr;
    char str[100];
    socdef = socket(AF_INET, SOCK_STREAM, 0);
    if (socdef == 0) {
        printf("socket creation failed");
        return -1;
    }
    memset(&addr, '0', sizeof(addr));
    addr.sin_family = AF_INET;
    addr.sin_port = htons(8086);
    if (inet_pton(AF_INET, "127.0.0.1", &addr.sin_addr) < 0) {
        printf("error");
        return -1;
    }
    if ((connect(socdef, (struct sockaddr *)&addr, sizeof(addr))) < 0) {
        printf("error in new socket creation");
        return -1;
    }
    printf("enter the string: ");
    scanf("%s", str);

    send(socdef, str, sizeof(str), 0);
    readval = read(socdef, str, sizeof(str));
    printf("reverse: %s", str);
}
```

**Server Side**

```
#include <stdio.h>
#include <stdlib.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <string.h>
#include <netinet/in.h>
int main() {
    int socdef, newsock, readval;
```

```

struct sockaddr_in addr;
char str[100], temp;
int addrlen = sizeof(addr);
socdef = socket(AF_INET, SOCK_STREAM, 0);
if (socdef == 0) {
    printf("socket creation failed");
    return 1;
}
addr.sin_family = AF_INET;
addr.sin_port = htons(8086);
addr.sin_addr.s_addr = INADDR_ANY;
if (bind(socdef, (struct sockaddr *)&addr, sizeof(addr)) < 0) {
    printf("binding failed");
    return -1;
}
printf("bind created\n");
if (listen(socdef, 3) < 0) {
    printf("client connection terminated");
    return -1;
}
printf("listening...\n");
if ((newsock = accept(socdef, (struct sockaddr *)&addr, (socklen_t *)&addrlen)) < 0) {
    printf("error in new socket creation");
    return -1;
}
printf("new socket created\n");
readval = read(newsock, str, sizeof(str));
int i, j, k;
k = strlen(str);
for (i = 0, j = k - 1; i < j; i++, j--) {
    temp = str[i];
    str[i] = str[j];
    str[j] = temp;
}
send(newsock, str, sizeof(str), 0);
}

```

### **OUTPUT:**

#### **Server Side:**

```

$ ./server
Bind created
Listening....
New socket created

```

#### **Client Side:**

```

$ ./client
Enter the string: hello
Reversed string from server: olleh

```



**Ex.No:4**

**IMPLEMENTATION OF CLIENT-SERVER COMMUNICATION USING  
SOCKET PROGRAMMING AND UDP AS TRANSPORT LAYER PROTOCOL**

**Progam:**

**Server Side:**

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <string.h>
#include <netinet/in.h>
#include <unistd.h>

void main() {
    int sockdef;
    struct sockaddr_in client, server;
    int clientlen = sizeof(client);
    int r = 2, c = 2, i, j, a[100][100], b[100][100], flat[100], k, result[100][100];
    sockdef = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);
    if (sockdef < 0) {
        printf("Socket creation failed\n");
        return;
    }
    server.sin_family = AF_INET;
    server.sin_port = htons(8090);
    server.sin_addr.s_addr = INADDR_ANY;

    if (bind(sockdef, (struct sockaddr*)&server, sizeof(server)) < 0) {
        printf("Binding failed\n");
        return;
    }
    // Receive first matrix
    if (recvfrom(sockdef, flat, sizeof(flat), 0, (struct sockaddr*)&client, &clientlen) < 0)
        printf("Receiving failed\n");

    k = 0;
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            a[i][j] = flat[k++];

    // Receive second matrix
    if (recvfrom(sockdef, flat, sizeof(flat), 0, (struct sockaddr*)&client, &clientlen) < 0)
        printf("Receiving failed\n");

    k = 0;
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            b[i][j] = flat[k++];
```

```

// Perform addition
k = 0;
printf("\nMatrix Addition Result (Server Side):\n");
for (i = 0; i < r; i++) {
    for (j = 0; j < c; j++) {
        result[i][j] = a[i][j] + b[i][j];
        printf("%d\t", result[i][j]);
        flat[k++] = result[i][j];
    }
    printf("\n");
}

// Send result back to client
if (sendto(sockdef, flat, sizeof(flat), 0, (struct sockaddr*)&client, clientlen) < 0) {
    printf("Sending failed\n");
    return;
}

printf("Result sent to client successfully.\n");
close(sockdef);
}

```

### **Client Side:**

```

#include <stdio.h>
#include <stdlib.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <string.h>
#include <netinet/in.h>
#include <unistd.h>

void main() {
    int sockdef;
    struct sockaddr_in server;
    int clientlen = sizeof(server);
    int r = 2, c = 2, i, j, a[100][100], b[100][100], flat[100], k;

    sockdef = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);
    if (sockdef < 0) {
        printf("Socket creation failed\n");
        return;
    }

    server.sin_family = AF_INET;
    server.sin_port = htons(8090);
    inet_pton(AF_INET, "127.0.0.1", &server.sin_addr);
}

```

```

// Input first matrix
printf("Enter elements for 1st matrix:\n");
for (i = 0; i < r; i++)
    for (j = 0; j < c; j++) {
        printf("[%d][%d]: ", i + 1, j + 1);
        scanf("%d", &a[i][j]);
    }

// Send first matrix
k = 0;
for (i = 0; i < r; i++)
    for (j = 0; j < c; j++)
        flat[k++] = a[i][j];

sendto(sockdef, flat, sizeof(flat), 0, (struct sockaddr*)&server, clientlen);

// Input second matrix
printf("Enter elements for 2nd matrix:\n");
for (i = 0; i < r; i++)
    for (j = 0; j < c; j++) {
        printf("[%d][%d]: ", i + 1, j + 1);
        scanf("%d", &b[i][j]);
    }

// Send second matrix
k = 0;
for (i = 0; i < r; i++)
    for (j = 0; j < c; j++)
        flat[k++] = b[i][j];

sendto(sockdef, flat, sizeof(flat), 0, (struct sockaddr*)&server, clientlen);

// Receive result
recvfrom(sockdef, flat, sizeof(flat), 0, (struct sockaddr*)&server, &clientlen);

printf("\nResult matrix received from server:\n");
k = 0;
for (i = 0; i < r; i++) {
    for (j = 0; j < c; j++)
        printf("%d\t", flat[k++]);
    printf("\n");
}

close(sockdef);
}

```

**OUTPUT:****Server Side:**

```
$ gcc server.c -o server
```

```
$ ./server
```

Matrix Addition Result (Server Side):

```
6  8
```

```
10 12
```

Result sent to client successfully.

**Client Side:**

```
$ gcc client.c -o client
```

```
$ ./client
```

Enter elements for 1st matrix:

```
[1][1]: 1
```

```
[1][2]: 2
```

```
[2][1]: 3
```

```
[2][2]: 4
```

Enter elements for 2nd matrix:

```
[1][1]: 5
```

```
[1][2]: 6
```

```
[2][1]: 7
```

```
[2][2]: 8
```

Result matrix received from server:

```
6  8
```

```
10 12
```

**Ex.No:5(a)**

**SIMULATION OF SLIDING WINDOW PROTOCOLS  
STOP AND WAIT PROTOCOL**

**PROGRAM:**

**Server Side**

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#define PORT 8080
#define BUFFER_SIZE 1024
int main() {
    int server_fd, new_socket;
    struct sockaddr_in address;
    int addrlen = sizeof(address);
    char buffer[BUFFER_SIZE] = { 0 };
    int frame;
    // Create socket file descriptor
    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
        perror("Socket failed");
        exit(EXIT_FAILURE);
    }
    // Bind the socket to the port
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);
    if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0)
    {
        perror("Bind failed");
        exit(EXIT_FAILURE);
    }
    // Listen for incoming connections
    if (listen(server_fd, 3) < 0)
```

```

{
perror("Listen failed");
exit(EXIT_FAILURE);
}
printf("Server is listening on port %d\n", PORT);
// Accept a connection
if((new_socket = accept(server_fd, (struct sockaddr *)&address, (socklen_t *)&addrlen)) < 0)
{
perror("Accept failed");
exit(EXIT_FAILURE);
}
printf("Connection established with client.\n");
// Simulate receiving frames using Stop-and-Wait protocol
while (1)
{
memset(buffer, 0, BUFFER_SIZE);
int valread = read(new_socket, buffer, BUFFER_SIZE);
if (valread <= 0)
{
printf("Connection closed by client.\n");
break;
}
sscanf(buffer, "%d", &frame);
printf("Server: Received frame %d\n", frame);
// Send acknowledgment
char ack[BUFFER_SIZE];
snprintf(ack, sizeof(ack), "ACK for frame %d", frame);
send(new_socket, ack, strlen(ack), 0);
printf("Server: Sent acknowledgment for frame %d\n", frame);
}
close(new_socket); close(server_fd); return 0;
}

```

### **Client Side:**

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080
#define BUFFER_SIZE 1024

int main()
{
    int sock;

    struct sockaddr_in serv_addr;
    char buffer[BUFFER_SIZE] = { 0 };
    int frame = 0, total_frames;

    // Create socket file descriptor
    if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0)
    {
        perror("Socket creation error");
        exit(EXIT_FAILURE);
    }

    serv_addr.sin_family = AF_INET; serv_addr.sin_port = htons(PORT);

    // Convert IPv4 and IPv6 addresses from text to binary form
    if (inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr) <= 0)
    {
        perror("Invalid address/ Address not supported");
        exit(EXIT_FAILURE);
    }

    // Connect to the server
    if (connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0)
    {
        perror("Connection Failed");
        exit(EXIT_FAILURE);
    }
}
```

```

}

printf("Enter the total number of frames to send: "); scanf("%d", &total_frames);
while (frame < total_frames)
{
    // Send frame
    snprintf(buffer, sizeof(buffer), "%d", frame); send(sock, buffer, strlen(buffer), 0);
    printf("Client: Sent frame %d\n", frame);
    // Wait for acknowledgment
    memset(buffer, 0, BUFFER_SIZE);
    int valread = read(sock, buffer, BUFFER_SIZE);
    if (valread > 0)
    {
        printf("Client: Received acknowledgment: %s\n", buffer);
        frame++;
    }
    else
    {
        printf("Client: No acknowledgment received. Retransmitting frame %d\n", frame);
    }
}

printf("All frames sent successfully.\n");
close(sock);
return 0;
}

```

### **Output:**

#### **Server**

```
exam@CSL01:~/nwlab$ gedit exp5aserver.c
```

```
exam@CSL01:~/nwlab$ gcc exp5aserver.c
```

```
exam@CSL01:~/nwlab$ ./a.out
```

Server is listening on port 8080

Connection established with client.



Server: Received frame 0  
Server: Sent acknowledgment for frame 0  
Server: Received frame 1  
Server: Sent acknowledgment for frame 1  
Server: Received frame 2  
Server: Sent acknowledgment for frame 2  
Connection closed by client.

### **Client**

```
exam@CSL01:~/nwlab$ gedit exp5aclient.c
exam@CSL01:~/nwlab$ gcc exp5aclient.c
exam@CSL01:~/nwlab$ ./a.out
```

Enter the total number of frames to send: 3

Client: Sent frame 0  
Client: Received acknowledgment: ACK for frame 0  
Client: Sent frame 1  
Client: Received acknowledgment: ACK for frame 1  
Client: Sent frame 2  
Client: Received acknowledgment: ACK for frame 2  
All frames sent successfully.

**PROGRAM:****Server side:**

```
#include <stdio.h>
#include <netdb.h>
#include <netinet/in.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <sys/time.h>
#include <unistd.h>
#define MAX 80
#define PORT 8080
#define SA struct sockaddr
struct timeval timeout;
void func(int connfd)
{
    char buff[MAX];
    int f, c, ack, next = 0;
    while (1)
    {
        sleep(1);
        bzero(buff, MAX);
        recv(connfd, buff, MAX, 0);
        if (strcmp("Exit", buff) == 0)
        {
            printf("Exit\n");
            break;
        }
        f = atoi(buff);
        if (f != next)
        {
            printf("Frame %d discarded\nAcknowledgement sent: %d\n", f, ack);
            bzero(buff, MAX);
            snprintf(buff, sizeof(buff), "%d", ack);
            send(connfd, buff, sizeof(buff), 0);
            continue;
        }
        c = rand() % 3;
        switch (c)
        {
            case 0:
                // printf("Frame %d not received\n",f);
                break;
            case 1:
```

```

        ack = f;
        sleep(2);
        printf("Frame %d received\nAcknowledement sent: %d\n", f, ack);
        bzero(buff, MAX);
        snprintf(buff, sizeof(buff), "%d", ack);
        send(connfd, buff, sizeof(buff), 0);
        next = ack + 1;
        break;
    case 2:
        ack = f;
        printf("Frame %d received\nAcknowledement sent: %d\n", f, ack);
        bzero(buff, MAX);
        snprintf(buff, sizeof(buff), "%d", ack);
        send(connfd, buff, sizeof(buff), 0);
        next = ack + 1;
        break;
    }
}
}
void main()
{
    int sockfd, connfd, len;
    struct sockaddr_in servaddr, cli;
    sockfd = socket(AF_INET, SOCK_STREAM, 0);
    if (sockfd == -1)
    {
        printf("Socket creation failed\n");
        exit(0);
    }
    else
        printf("Socket successfully created\n");
    bzero(&servaddr, sizeof(servaddr));
    servaddr.sin_family = AF_INET;
    servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
    servaddr.sin_port = htons(PORT);
    if ((bind(sockfd, (SA *)&servaddr, sizeof(servaddr))) != 0)
    {
        printf("socket bind failed\n");
        exit(0);
    }
    else
        printf("Socket successfully binded\n");
    if ((listen(sockfd, 5)) != 0)
    {
        printf("Listen failed\n");
        exit(0);
    }
    else
        printf("Server listening\n");
    len = sizeof(cli);

```

```

connfd = accept(sockfd, (SA *)&cli, &len);
if (connfd < 0)
{
    printf("Server accept failed\n");
    exit(0);
}
else
    printf("Server accept the client\n");
func(connfd);
close(sockfd);
}

```

### Client Side:

```

#include <netdb.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <time.h>
#include <sys/time.h>
#include <unistd.h>
#include <arpa/inet.h>
#define MAX 80
#define PORT 8080
#define SA struct sockaddr
struct timeval timeout;
void func(int sockfd, int nf, int ws)
{
    char buff[MAX];
    int ack, i = 0, n, k, w1 = 0, w2 = ws - 1, j, flag = 0;
    if (setsockopt(sockfd, SOL_SOCKET, SO_RCVTIMEO, (const char *)&timeout,
sizeof(timeout)) < 0)
        perror("setsockopt(SO_RCVTIMEO) failed");

    for (i = 0; i < nf && i <= w2; i++)
    {
        bzero(buff, sizeof(buff));
        snprintf(buff, sizeof(buff), "%d", i);
        k = send(sockfd, buff, sizeof(buff), 0);
        printf("Frame %d sent\n", i);
    }
    while (1)
    {
        if (w2 - w1 != ws - 1 && flag == 0 && i != nf)
        {
            bzero(buff, sizeof(buff));
            snprintf(buff, sizeof(buff), "%d", i);
            k = send(sockfd, buff, sizeof(buff), 0);
            printf("Frame %d sent\n", i);
            w2++;

```

```

        i++;
    }
    flag = 0;
    bzero(buff, sizeof(buff));
    n = recv(sockfd, buff, MAX, 0);
    ack = atoi(buff);
    if (n > 0)
    {
        if (ack + 1 == nf)
        {
            printf("Acknowledgement received: %d\nExit\n", ack);
            bzero(buff, sizeof(buff));
            strcpy(buff, "Exit");
            k = send(sockfd, buff, sizeof(buff), 0);
            break;
        }
        if (ack == w1)
        {
            w1++;
            printf("Acknowledgement received: %d\n", ack);
        }
    }
    else
    {
        printf("Acknowledgement not received for %d\nResending frames\n", w1);
        for (j = w1; j < nf && j < w1 + ws; j++)
        {
            bzero(buff, sizeof(buff));
            sprintf(buff, "%d", j);
            k = send(sockfd, buff, sizeof(buff), 0);
            printf("Frame %d sent\n", j);
        }
        flag = 1;
    }
}
}

void main()
{
    int sockfd, connfd, f, w;
    struct sockaddr_in servaddr, cli;
    sockfd = socket(AF_INET, SOCK_STREAM, 0);
    if (sockfd == -1)
    {
        printf("Socket creation failed\n");
        exit(0);
    }
    else
        printf("Socket successfully created\n");
    bzero(&servaddr, sizeof(servaddr));
    servaddr.sin_family = AF_INET;

```

```

servaddr.sin_addr.s_addr = inet_addr("127.0.0.1");
servaddr.sin_port = htons(PORT);
timeout.tv_sec = 3;
timeout.tv_usec = 0;
if (connect(sockfd, (SA *)&servaddr, sizeof(servaddr)) != 0)
{
    printf("Connection with the server failed\n");
    exit(0);
}
else
    printf("Connected to the server\n");
printf("Enter the number of frames: ");
scanf("%d", &f);
printf("Enter the window size: ");
scanf("%d", &w);
func(sockfd, f, w);
close(sockfd);
}

```

## OUTPUT:

### Server Side:

exam@CSL01:~/nwlab\$ gedit gbn\_server.c

exam@CSL01:~/nwlab\$ gcc gbn\_server.c

exam@CSL01:~/nwlab\$ ./a.out

Socket successfully created

Socket successfully binded

Server listening

Server accept the client

Frame 0 received

Acknowledgement sent: 0

Frame 1 received

Acknowledgement sent: 1

Frame 3 discarded

Acknowledgement sent: 1

Frame 4 discarded

Acknowledgement sent: 1

Frame 5 discarded

Acknowledgement sent: 1

Frame 2 received

Acknowledgement sent: 2

Frame 3 received

Acknowledgement sent: 3

Frame 4 received

Acknowledgement sent: 4

Frame 5 received

Acknowledgement sent: 5

Exit

**Client Side:**

```
exam@CSL01:~/nwlab$ gedit gbn_client.c
```

```
exam@CSL01:~/nwlab$ gcc gbn_client.c
```

```
exam@CSL01:~/nwlab$ ./a.out
```

Socket successfully created

Connected to the server

Enter the number of frames: 6

Enter the window size: 4

Frame 0 sent

Frame 1 sent

Frame 2 sent

Frame 3 sent

Acknowledgement received: 0

Frame 4 sent

Acknowledgement received: 1

Frame 5 sent

Acknowledgement not received for 2

Resending frames

Frame 2 sent

Frame 3 sent

Frame 4 sent

Frame 5 sent

Acknowledgement received: 2

Acknowledgement received: 3

Acknowledgement received: 4

Acknowledgement received: 5

Exit

Ex.No:5(c)

## SELECTIVE REPEAT ARQ PROTOCOL

### PROGRAM:

#### Server Side:

```
#include <netdb.h>
#include <netinet/in.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <unistd.h>
#include <arpa/inet.h>

void func(int connfd)
{
    char buff[80];
    int f, c, win, ack, k;
    win = 8;
    read(connfd, buff, sizeof(buff));
    if (strcmp("end", buff) == 0) {
        printf("Exit\n");
    }
    f = atoi(buff);
    f = atoi(buff);
    for (k = 0; k < f; k++)
        printf("\n%d ", k);
    c = rand() % (win - 1);
    c = rand() % (win - 1);
    for (k = 0; k < win; k++) {
        if (k != c) {
            sleep(1);
            printf("\nRecieved Frame: %d\n", k);
        }
    }
    sleep(c);
    printf("\nFrame %d not received\nWaiting for Frame %d\n", c, c);
    ack = c;
    snprintf(buff, sizeof(buff), "%d", ack);
    write(connfd, buff, sizeof(buff));
    read(connfd, buff, sizeof(buff));
    if (strcmp("end", buff) == 0) {
        printf("Exit\n");
    }
    f = atoi(buff);
    // for(k = f-1 ; k < win ; k++ )
    //{
    sleep(1);
    printf("\nRecieved Frame %d\n", (f - 1));
}
```



```

//}
printf("\n");
strcpy(buff, "end");
write(connfd, buff, sizeof(buff));
}

int main() {
    int sockfd, connfd, len;
    struct sockaddr_in servaddr, cli;
    sockfd = socket(AF_INET, SOCK_STREAM, 0);
    if (sockfd == -1) {
        printf("Socket creation failed\n");
        exit(0);
    }
    printf("Socket successfully created\n");
    servaddr.sin_family = AF_INET;
    servaddr.sin_addr.s_addr = inet_addr("127.0.0.1");
    servaddr.sin_port = htons(8080);
    if ((bind(sockfd, (struct sockaddr *) & servaddr, sizeof(servaddr))) != 0)
    {
        printf("socket bind failed\n");
        exit(0);
    }
    printf("Socket successfully binded\n");
    if ((listen(sockfd, 5)) != 0) {
        printf("Listen failed\n");
        exit(0);
    }
    printf("Server listening\n");
    len = sizeof(cli);
    connfd = accept(sockfd, (struct sockaddr *) & cli, & len);
    if (connfd < 0) {
        printf("Server accept failed\n");
        exit(0);
    }
    printf("Server accept the client\n");
    func(connfd);
    close(sockfd);
}

```

### **Client Side:**

```

#include <netdb.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <time.h>
#include <unistd.h>
#include <arpa/inet.h>

```

```

void func(int sockfd, int nf, int ws)
{
    char buff[80];
    int ack, i = 0, k, f;
    snprintf(buff, sizeof(buff), "%d", i);
    write(sockfd, buff, sizeof(buff));
    read(sockfd, buff, sizeof(buff));
    ack = atoi(buff);
    printf("Frames Sending : ");
    for (k = 0; k < 8; k++) {
        printf("%d ", k);
    }
    printf("\nFrame %d not sent properly. \n", ack);
    printf("Resending Frame : ");
    // for(k = ack ; k < 8 ; k++ )
    printf("%d ", ack);
    printf("\n");
    snprintf(buff, sizeof(buff), "%d", ack + 1);
    write(sockfd, buff, sizeof(buff));
    read(sockfd, buff, sizeof(buff));
    if (strcmp("end", buff) == 0) {
        printf("All frames sent successfully\n");
        printf("Exit\n");
    }
}

int main() {
    int sockfd, connfd, f, w;
    struct sockaddr_in servaddr, cli;
    sockfd = socket(AF_INET, SOCK_STREAM, 0);
    if (sockfd == -1) {
        printf("Socket creation failed\n");
        exit(0);
    } else
        printf("Socket successfully created\n");
    servaddr.sin_family = AF_INET;
    servaddr.sin_addr.s_addr = inet_addr("127.0.0.1");
    servaddr.sin_port = htons(8080);
    if (connect(sockfd, (struct sockaddr * ) & servaddr, sizeof(servaddr)) != 0) {
        printf("Connection with the reciever failed\n");
        exit(0);
    } else
        printf("Connected to the reciever\n");
    f = 0;
    func(sockfd, f, 1);
    close(sockfd);
}

```

## **Output:**

### **Server Side:**

```
exam@CSL01:~/nwlab$ gedit sr_server.c
exam@CSL01:~/nwlab$ gcc sr_server.c
exam@CSL01:~/nwlab$ ./a.out
Socket successfully created
Socket successfully binded
Server listening
Server accept the client
```

Recieved Frame: 0

Recieved Frame: 1

Recieved Frame: 2

Recieved Frame: 3

Recieved Frame: 5

Recieved Frame: 6

Recieved Frame: 7

Frame 4 not received  
Waiting for Frame 4

Recieved Frame 4

### **Client Side:**

```
exam@CSL01:~/nwlab$ gedit sr_client.c
exam@CSL01:~/nwlab$ gcc sr_client.c
exam@CSL01:~/nwlab$ ./a.out
Socket successfully created
Connected to the reciever
Frames Sending : 0 1 2 3 4 5 6 7
Frame 4 not sent properly.
Resending Frame : 4
All frames sent successfully
Exit
```

**Ex.No:6(a)**

## **DISTANCE VECTOR ROUTING PROTOCOL**

### **Program:**

```
#include <stdio.h>

#define MAX 20

struct node {
    unsigned from[MAX];
    unsigned dist[MAX];
} rt[MAX];

void main() {
    int i, j, n, cost[MAX][MAX], count, k;

    printf("Enter number of nodes (max %d): ", MAX);
    scanf("%d", &n);

    if (n <= 0 || n > MAX) {
        printf("Invalid number of nodes.\n");
        return;
    }

    printf("Use 999 to represent infinity.\n");
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            printf("Cost from node %d to node %d: ", i + 1, j + 1);
            scanf("%d", &cost[i][j]);
            rt[i].dist[j] = cost[i][j];
            rt[i].from[j] = j;
        }
        cost[i][i] = 0;
        rt[i].dist[i] = 0;
        rt[i].from[i] = i;
    }
```

```

do {
    count = 0;
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            for (k = 0; k < n; k++) {
                if (rt[i].dist[j] > cost[i][k] + rt[k].dist[j]) {
                    rt[i].dist[j] = cost[i][k] + rt[k].dist[j];
                    rt[i].from[j] = k;
                    count++;
                }
            }
        }
    }
} while (count != 0);

printf("\n--- Routing Tables ---\n");
for (i = 0; i < n; i++) {
    printf("\nRouting table for router %d:\n", i + 1);
    printf("Destination\tNext Hop\tDistance\n");
    for (j = 0; j < n; j++) {
        printf("%d\t%d\t%d\n", j + 1, rt[i].from[j] + 1, rt[i].dist[j]);
    }
}
}

```

## **OUTPUT:**

### **Sample Input:**

Enter number of nodes (max 20): 3

Use 999 to represent infinity.

Cost from node 1 to node 1: 0

Cost from node 1 to node 2: 2

Cost from node 1 to node 3: 7

Cost from node 2 to node 1: 2

Cost from node 2 to node 2: 0

Cost from node 2 to node 3: 1

Cost from node 3 to node 1: 7

Cost from node 3 to node 2: 1

Cost from node 3 to node 3: 0

### **Sample Output:**

--- Routing Tables ---

Routing table for router 1:

| Destination | Next Hop | Distance |
|-------------|----------|----------|
| 1           | 1        | 0        |
| 2           | 2        | 2        |
| 3           | 2        | 3        |

Routing table for router 2:

| Destination | Next Hop | Distance |
|-------------|----------|----------|
| 1           | 1        | 2        |
| 2           | 2        | 0        |
| 3           | 3        | 1        |

Routing table for router 3:

| Destination | Next Hop | Distance |
|-------------|----------|----------|
| 1           | 2        | 3        |
| 2           | 2        | 1        |
| 3           | 3        | 0        |

**Ex.No: 6(b)**

**LINK STATE ROUTING (DIJKSTRA'S ALGORITHM)**

**Program:**

```
#include <stdio.h>

#define INFINITY 9999
#define MAX 10

void dijkstra(int G[MAX][MAX], int n, int startnode);

int main() {
    int G[MAX][MAX], i, j, n, u;
    printf("Enter no. of vertices: ");
    scanf("%d", &n);
    printf("\nEnter the adjacency matrix:\n");
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            scanf("%d", &G[i][j]);
    printf("\nEnter the starting node: ");
    scanf("%d", &u);
    dijkstra(G, n, u);
    return 0;
}

void dijkstra(int G[MAX][MAX], int n, int startnode) {
    int cost[MAX][MAX], distance[MAX], pred[MAX];
    int visited[MAX], count, mindistance, nextnode, i, j;

    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            if (G[i][j] == 0)
                cost[i][j] = INFINITY;
            else
                cost[i][j] = G[i][j];
```

```

for (i = 0; i < n; i++) {
    distance[i] = cost[startnode][i];
    pred[i] = startnode;
    visited[i] = 0;
}

```

```

distance[startnode] = 0;
visited[startnode] = 1;
count = 1;

```

```

while (count < n - 1) {
    mindistance = INFINITY;
    for (i = 0; i < n; i++)
        if (distance[i] < mindistance && !visited[i]) {
            mindistance = distance[i];
            nextnode = i;
        }

```

```

    visited[nextnode] = 1;
    for (i = 0; i < n; i++)
        if (!visited[i])
            if (mindistance + cost[nextnode][i] < distance[i]) {
                distance[i] = mindistance + cost[nextnode][i];
                pred[i] = nextnode;
            }
    count++;
}

```

```

for (i = 0; i < n; i++)
    if (i != startnode) {
        printf("\nDistance of node %d = %d", i, distance[i]);
        printf("\nPath = %d", i);
    }

```



```

        j = i;
        do {
            j = pred[j];
            printf(" <- %d", j);
        } while (j != startnode);
    }
}

```

### **OUTPUT:**

#### **Sample Input:**

Enter no. of vertices: 4

Enter the adjacency matrix:

0 2 4 0

2 0 1 7

4 1 0 3

0 7 3 0

Enter the starting node: 0

#### **Sample Output:**

Distance of node 1 = 2

Path = 1 <- 0

Distance of node 2 = 3

Path = 2 <- 1 <- 0

Distance of node 3 = 6

Path = 3 <- 2 <- 1 <- 0

**Ex.No:7**

## **IMPLEMENTATION OF SMTP**

### **Program:**

#### **Client Side:**

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <netdb.h>
#include <netinet/in.h>
#include <sys/types.h>
#include <sys/socket.h>

#define SA struct sockaddr
#define PORT 8086

void main() {
    int sockfd;
    struct sockaddr_in servaddr;
    sockfd = socket(AF_INET, SOCK_STREAM, 0);
    if (sockfd < 0) {
        printf("Socket creation failed.\n");
        exit(0);
    }
    printf("Socket created successfully.\n");

    servaddr.sin_family = AF_INET;
    servaddr.sin_addr.s_addr = inet_addr("127.0.0.1");
    servaddr.sin_port = htons(PORT);

    if (connect(sockfd, (SA *)&servaddr, sizeof(servaddr)) < 0) {
```

```

    printf("Connection failed.\n");
    exit(0);
}
printf("Connection successful.\n");

char command[100];
char sender_mail_addr[100];
char recipient_mail_addr[100];
char body[200];

printf("Enter HELO command to start communication with server: ");
fgets(command, sizeof(command), stdin);
write(sockfd, command, sizeof(command));

if (strcmp(command, "HELO\n") == 0)
    printf("HELO command sent.\nCommunication established.\n");

while (1) {
    printf("\n1. MAIL FROM - Define sender mail address\n");
    printf("2. RCPT TO - Define receiver's mail address\n");
    printf("3. DATA - Define body of the mail\n");
    printf("4. QUIT\n");
    printf("Enter a command: ");
    fgets(command, sizeof(command), stdin);
    write(sockfd, command, sizeof(command));

    if (strcmp(command, "MAIL FROM\n") == 0) {
        printf("Enter sender's mail address: ");
        fgets(sender_mail_addr, sizeof(sender_mail_addr), stdin);
        write(sockfd, sender_mail_addr, sizeof(sender_mail_addr));
    } else if (strcmp(command, "RCPT TO\n") == 0) {
        printf("Enter receiver's mail address: ");

```

```

    fgets(recipient_mail_addr, sizeof(recipient_mail_addr), stdin);
    write(sockfd, recipient_mail_addr, sizeof(recipient_mail_addr));
} else if (strcmp(command, "DATA\n") == 0) {
    printf("Enter mail body: ");
    fgets(body, sizeof(body), stdin);
    write(sockfd, body, sizeof(body));
} else if (strcmp(command, "QUIT\n") == 0)
    exit(0);
else
    printf("Invalid command.\n");

if (strcmp(sender_mail_addr, "") != 0 &&
    strcmp(recipient_mail_addr, "") != 0 &&
    strcmp(body, "") != 0) {
    printf("Mail sent.\n");
    strcpy(sender_mail_addr, "");
    strcpy(recipient_mail_addr, "");
    strcpy(body, "");
}
}
}

```

### **Server Side:**

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <netdb.h>
#include <netinet/in.h>
#include <sys/types.h>
#include <sys/socket.h>

```

```
#define SA struct sockaddr

#define PORT 8086

void main() {
    int sockfd, connfd, len;
    struct sockaddr_in servaddr, cliaddr;

    sockfd = socket(AF_INET, SOCK_STREAM, 0);
    if (sockfd < 0) {
        printf("Socket creation failed.\n");
        exit(0);
    }
    printf("Socket created successfully.\n");

    servaddr.sin_family = AF_INET;
    servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
    servaddr.sin_port = htons(PORT);

    if (bind(sockfd, (SA *)&servaddr, sizeof(servaddr)) < 0) {
        printf("Bind failed.\n");
        exit(0);
    }
    printf("Bind successful.\n");

    if (listen(sockfd, 5) < 0) {
        printf("Listen failed.\n");
        exit(0);
    }
    printf("Listen successful.\n");

    len = sizeof(cliaddr);
    connfd = accept(sockfd, (SA *)&cliaddr, &len);
```

```

if (connfd < 0) {
    printf("Connection failed.\n");
    exit(0);
}
printf("Connection successful.\n");

char command[100];
char sender_mail_addr[100];
char recipient_mail_addr[100];
char body[200];

printf("Enter HELO command at client side to start communication.\n");
read(connfd, command, sizeof(command));

if (strcmp(command, "HELO\n") == 0)
    printf("HELO command received.\nCommunication established.\n");

while (1) {
    read(connfd, command, sizeof(command));
    if (strcmp(command, "MAIL FROM\n") == 0)
        read(connfd, sender_mail_addr, sizeof(sender_mail_addr));
    else if (strcmp(command, "RCPT TO\n") == 0)
        read(connfd, recipient_mail_addr, sizeof(recipient_mail_addr));
    else if (strcmp(command, "DATA\n") == 0)
        read(connfd, body, sizeof(body));
    else if (strcmp(command, "QUIT\n") == 0)
        exit(0);
    else
        printf("Invalid command.\n");

    if (strcmp(sender_mail_addr, "") != 0 &&
        strcmp(recipient_mail_addr, "") != 0 &&

```

```

        strcmp(body, "") != 0) {
        printf("\nMail received.\n");
        printf("Sender mail address: %s", sender_mail_addr);
        printf("Recipient mail address: %s", recipient_mail_addr);
        printf("Body:\n%s\n", body);
        strcpy(sender_mail_addr, "");
        strcpy(recipient_mail_addr, "");
        strcpy(body, "");
    }
}
}

```

## **OUTPUT:**

### **Server Side:**

\$ ./server

Socket created successfully.

Bind successful.

Listen successful.

Connection successful.

Enter HELO command at client side to start communication.

HELO command received.

Communication established.

Mail received.

Sender mail address: alice@example.com

Recipient mail address: bob@example.com

Body:

Hello Bob,

This is a test mail using SMTP simulation.

### **Client Side:**

\$ ./client

Socket created successfully.

Connection successful.

Enter HELO command to start communication with server: HELO

HELO command sent.

Communication established.

1. MAIL FROM - Define sender mail address

2. RCPT TO - Define receiver's mail address

3. DATA - Define body of the mail

4. QUIT

Enter a command: MAIL FROM

Enter sender's mail address: alice@example.com

Enter a command: RCPT TO

Enter receiver's mail address: bob@example.com

Enter a command: DATA

Enter mail body: Hello Bob,

This is a test mail using SMTP simulation.

Mail sent.

Enter a command: QUIT



**Ex.No:8**

## **IMPLEMENTATION OF FILE TRANSFER PROTOCOL**

### **Program:**

#### **Client Side:**

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <fcntl.h>
#include <string.h>
#include <netdb.h>
#include <stdlib.h>

void main() {
    struct sockaddr_in serveraddr;
    int clientsock, n, rdret, length;
    char filename[20], filedata[300];

    bzero((char *)&serveraddr, sizeof(serveraddr));
    serveraddr.sin_family = AF_INET;
    serveraddr.sin_port = htons(8086);
    serveraddr.sin_addr.s_addr = inet_addr("127.0.0.1");

    clientsock = socket(AF_INET, SOCK_STREAM, 0);
    if (clientsock < 0) {
        perror("Socket creation failed");
        exit(1);
    }
}
```

```

if (connect(clientsock, (struct sockaddr *)&serveraddr, sizeof(serveraddr)) < 0) {
    printf("\nError: Cannot connect...\n");
    exit(0);
} else
    printf("Socket created successfully.\n");

printf("Enter the name of the file : ");
scanf("%s", filename);
length = strlen(filename);

write(clientsock, filename, length);

rdret = read(clientsock, filedata, 300);
printf("\nThe contents of the file:\n%s\n", filedata);

close(clientsock);
}

```

### **Server Side:**

```

#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <fcntl.h>
#include <string.h>
#include <stdlib.h>

void main() {
    struct sockaddr_in clientaddr, serveraddr;
    int serversock, newserversock, clientsize, n, f;
    char filename[100], filedata[300];

```

```

serversock = socket(AF_INET, SOCK_STREAM, 0);
if (serversock < 0) {
    perror("Socket creation failed");
    exit(1);
}
printf("Server socket created.\n");

bzero((char *)&serveraddr, sizeof(serveraddr));
serveraddr.sin_family = AF_INET;
serveraddr.sin_port = htons(8086);
serveraddr.sin_addr.s_addr = inet_addr("127.0.0.1");

if (bind(serversock, (struct sockaddr *)&serveraddr, sizeof(serveraddr)) < 0) {
    perror("Bind failed");
    exit(1);
}
printf("Binding successful.\n");

if (listen(serversock, 5) < 0) {
    printf("Listen failed.\n");
    exit(0);
} else
    printf("Listen successful.\n");

clientsize = sizeof(clientaddr);
newserversock = accept(serversock, (struct sockaddr *)&clientaddr, &clientsize);
if (newserversock < 0) {
    printf("Connection failed.\n");
    exit(0);
}
printf("Connection successful.\n");

```

```

n = read(newserversock, filename, sizeof(filename));
filename[n] = '\0';
printf("\nThe requested file from the client is %s.\n", filename);

f = open(filename, O_RDWR);
if (f < 0) {
    perror("File open failed");
    strcpy(filedata, "File not found on server.");
    write(newserversock, filedata, strlen(filedata));
} else {
    n = read(f, filedata, sizeof(filedata) - 1);
    filedata[n] = '\0';
    printf("\nThe contents of the file:\n%s\n", filedata);
    write(newserversock, filedata, strlen(filedata));
    close(f);
}

close(serversock);
close(newserversock);
}

```

## **OUTPUT:**

### **Sample.txt**

Welcome to the File Transfer Protocol Experiment!

This is a test file.

### **Server Side:**

\$ ./server

Server socket created.

Binding successful.

Listen successful.

Connection successful.

The requested file from the client is sample.txt.

The contents of the file:

Welcome to the File Transfer Protocol Experiment!

This is a test file.

**Client Side:**

\$ ./client

Socket created successfully.

Enter the name of the file : sample.txt

The contents of the file:

Welcome to the File Transfer Protocol Experiment!

This is a test file.

**Ex.No:9**

## **CONGESTION CONTROL USING LEAKY BUCKET ALGORITHM**

### **Program:**

```
#include <stdio.h>

void main() {
    int in, out, size, n, store = 0;

    printf("Enter bucket size: ");
    scanf("%d", &size);

    printf("Enter number of inputs: ");
    scanf("%d", &n);

    printf("Enter outgoing rate: ");
    scanf("%d", &out);

    while (n != 0) {
        printf("Incoming packet size: ");
        scanf("%d", &in);

        if (in <= (size - store)) {
            store += in;
            printf("Bucket buffer size %d out of %d\n", store, size);
        } else {
            printf("Dropped %d number of packets\n", in - (size - store));
            store = size;
            printf("Bucket buffer size %d out of %d\n", store, size);
        }

        store -= out;
        if (store < 0)
            store = 0;
    }
}
```

```
    printf("After outgoing %d packets left out of %d in buffer\n", store, size);  
    n--;  
}  
}
```

### **OUTPUT:**

Enter bucket size: 10

Enter number of inputs: 4

Enter outgoing rate: 3

Incoming packet size: 4

Bucket buffer size 4 out of 10

After outgoing 1 packets left out of 10 in buffer

Incoming packet size: 6

Bucket buffer size 7 out of 10

After outgoing 4 packets left out of 10 in buffer

Incoming packet size: 8

Dropped 2 number of packets

Bucket buffer size 10 out of 10

After outgoing 7 packets left out of 10 in buffer

Incoming packet size: 5

Bucket buffer size 10 out of 10

After outgoing 7 packets left out of 10 in buffer

**Aim:**

To study and understand the working of Wireshark, a network protocol analyzer tool used to capture and analyze network traffic.

**Theory:**

Wireshark is an open-source network analysis tool that captures packets traveling through a network in real time. It is widely used for:

- Network troubleshooting
- Protocol analysis
- Security and forensic investigations
- Learning network communication

Wireshark captures packets and displays detailed information about their structure, including:

- Source and destination addresses
- Protocol used (TCP, UDP, ICMP, HTTP, etc.)
- Port numbers
- Payload data

**Features of Wireshark:**

1. Real-time packet capture and offline analysis.
2. Supports hundreds of network protocols.
3. Color-coding for quick packet identification.
4. Display filters for specific traffic (e.g., HTTP, DNS).
5. Ability to reconstruct TCP sessions.
6. Export capture data in multiple formats (.pcap, .csv).

**Procedure:****Step 1: Launch Wireshark**

- Open Wireshark from the desktop or start menu.
- The main window shows a list of available network interfaces (e.g., Ethernet, Wi-Fi).

**Step 2: Start Capture**

- Select an active network interface.
- Click on Start Capturing Packets (blue shark fin icon).
- Wireshark begins capturing packets in real time.



### Step 3: Observe Captured Packets

Each line in the capture window displays:

- Packet number
- Time
- Source and Destination IP
- Protocol used
- Packet information

### Step 4: Apply Filters

You can use filters to narrow down the packets displayed.

| Filter                      | Description                    |
|-----------------------------|--------------------------------|
| <code>ip</code>             | Displays all IP packets        |
| <code>http</code>           | Displays only HTTP packets     |
| <code>dns</code>            | Displays DNS queries/responses |
| <code>icmp</code>           | Displays ping packets          |
| <code>tcp.port == 80</code> | Shows packets on TCP port 80   |

### Step 5: Analyze a Packet

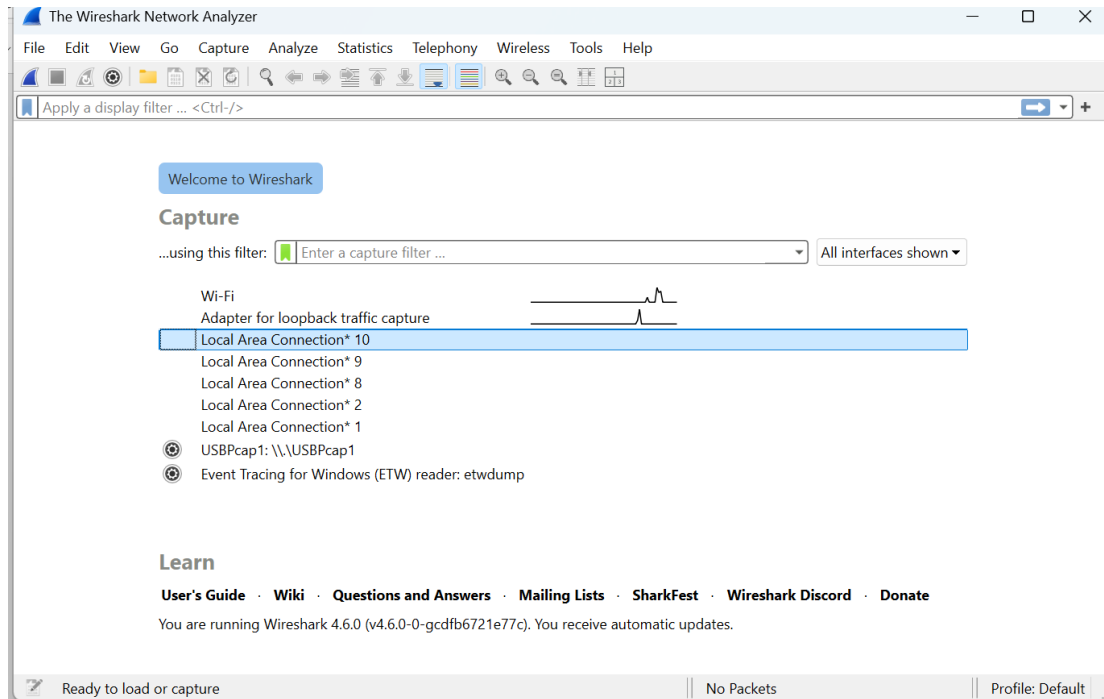
- Click any packet to view details in the Packet Details Pane.
- You can explore:
  - Frame: Basic information (time, size)
  - Ethernet: MAC addresses
  - IP: Source/Destination IP
  - TCP/UDP: Port numbers
  - Application layer: HTTP, DNS, etc.

### Step 6: Stop Capture

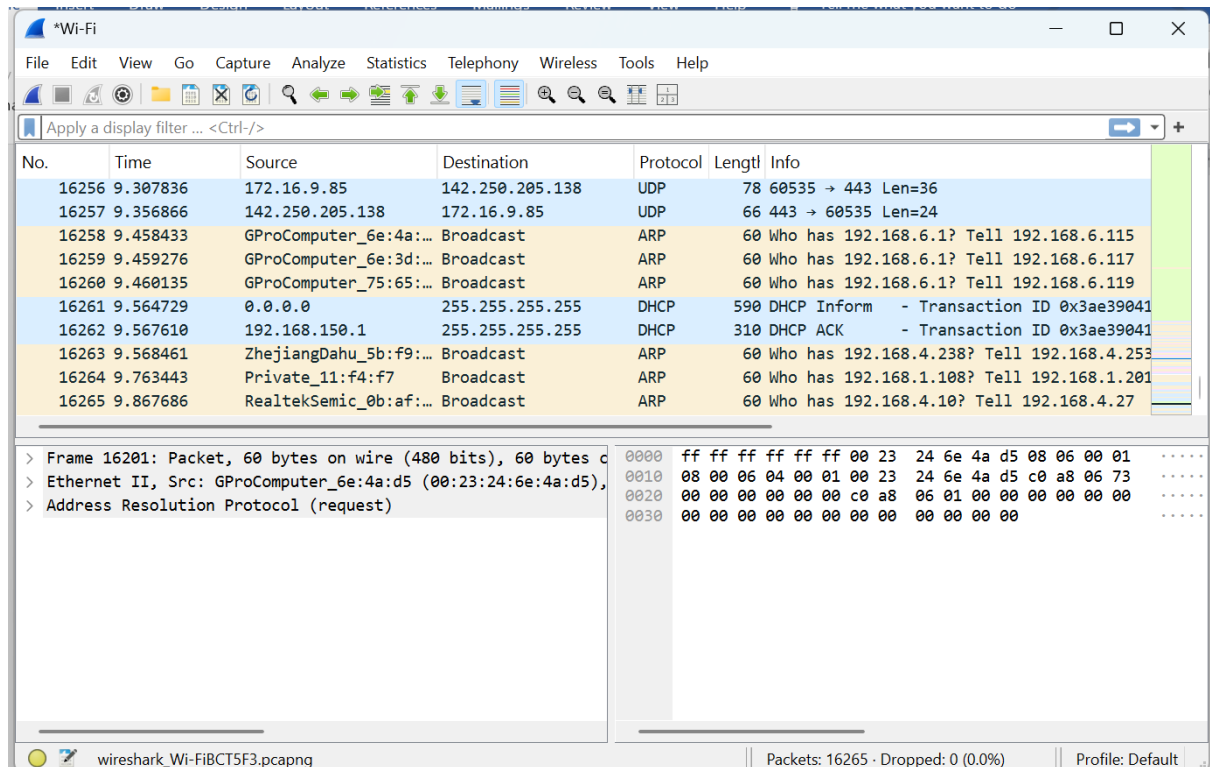
- Click the red square icon to stop.
- Go to File → Save As to store your capture file (.pcapng).

## Sample Output:

### Wireshark main Interface:



### Wireshark Working Screen:



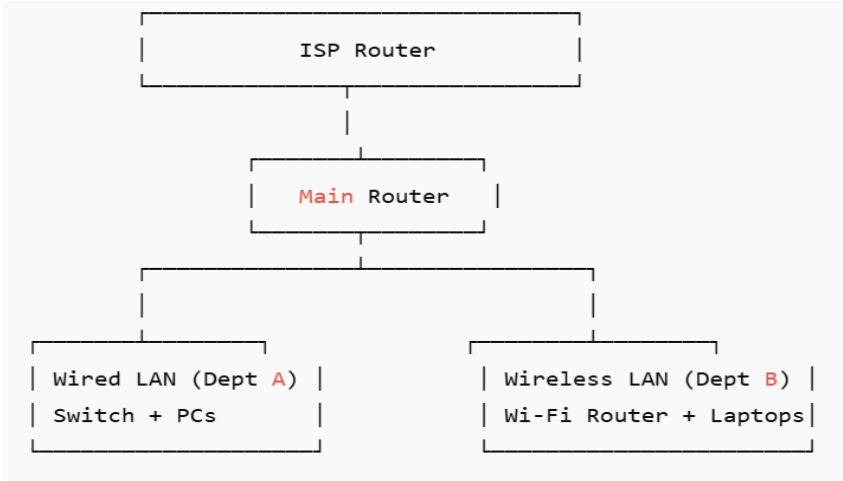
Ex.No.11

NETWORK DESIGN AND CONFIGURATION WITH MULTIPLE SUBNETS  
(WIRED & WIRELESS LANS)

Aim:

To design and configure a network with multiple subnets consisting of both wired and wireless LANs, and to configure commonly used network services such as DHCP, DNS, and HTTP using appropriate network devices.

Network Topology Overview:



IP Addressing Scheme

| Subnet                  | Network Address | Range                           | Default Gateway | Devices               |  |
|-------------------------|-----------------|---------------------------------|-----------------|-----------------------|--|
| Subnet 1 (Wired LAN)    | 192.168.10.0/24 | 192.168.10.1–<br>192.168.10.254 | 192.168.10.1    | PCs, Switch           |  |
| Subnet 2 (Wireless LAN) | 192.168.20.0/24 | 192.168.20.1–<br>192.168.20.254 | 192.168.20.1    | Laptops, Wi-Fi Router |  |
| Server Network          | 192.168.30.0/24 | 192.168.30.1–<br>192.168.30.254 | 192.168.30.1    | DHCP, DNS, Web Server |  |

Steps for Configuration

Step 1: Network Design

1. Place the following devices:
- 1 Router (e.g., Cisco 2811)

○ 2 Switches

○ 1 Wireless Router

○ 3 PCs (Wired)

○ 2 Laptops (Wireless)

○ 1 Server

2. Connect:

- Router → Switch1 (FastEthernet 0/0)
- Router → Wireless Router (FastEthernet 0/1)
- PCs to Switch1 using straight cables
- Laptops connect wirelessly to the Wireless Router

## Step 2: Configure IP Addresses

### Router Configuration

```
Router> enable
Router# configure terminal
Router(config)# interface fa0/0
Router(config-if)# ip address 192.168.10.1 255.255.255.0
Router(config-if)# no shutdown
Router(config)# interface fa0/1
Router(config-if)# ip address 192.168.20.1 255.255.255.0
Router(config-if)# no shutdown
Router(config)# interface fa1/0
Router(config-if)# ip address 192.168.30.1 255.255.255.0
Router(config-if)# no shutdown
Router(config)# exit
```

## Step 3: Configure DHCP Server (on Server Device)

1. Go to **Server** → **Services** → **DHCP**
2. Turn DHCP **ON**
3. Add the following pools:

### For Wired LAN

Pool Name: WiredLAN  
Default Gateway: 192.168.10.1  
DNS Server: 192.168.30.2  
Starting IP: 192.168.10.10  
Subnet Mask: 255.255.255.0  
Maximum Users: 20

## **For Wireless LAN**

Pool Name: WirelessLAN

Default Gateway: 192.168.20.1

DNS Server: 192.168.30.2

Starting IP: 192.168.20.10

Subnet Mask: 255.255.255.0

Maximum Users: 20

## **Step 4: Configure DNS and HTTP Service**

### **DNS Configuration**

- Go to **Server** → **Services** → **DNS**
- Turn DNS **ON**
- Add entry:

Name: www.college.local

Address: 192.168.30.2

### **HTTP Configuration**

- Go to **Server** → **Services** → **HTTP**
- Turn HTTP **ON**
- Add a simple webpage in the “index.html” field:

<h2>Welcome to AISAT College Network</h2>

## **Step 5: Configure Wireless Router**

1. Assign IP Address: 192.168.20.2
2. Set SSID: AISAT\_WIFI
3. Configure WPA2 security and password (e.g., aisat123)
4. Connect laptops using this SSID.

## **Step 6: Client Configuration**

- PCs and laptops get IP addresses automatically from DHCP.
- Verify using:

PC> ipconfig

## **Step 7: Verification**

### **1. Ping Test**

2. PC> ping 192.168.20.10

(Check inter-subnet connectivity)

### **3. DNS Test**

4. PC> ping www.college.local

(DNS should resolve to 192.168.30.2)

### **5. HTTP Test**

- Open PC Web Browser → Enter: www.college.local
- Webpage should display: “Welcome to AISAT College Network”

**Aim:**

To study and understand the working of the **Network Simulator 2 (NS2)** tool used for simulating and analyzing network protocols and communication behavior.

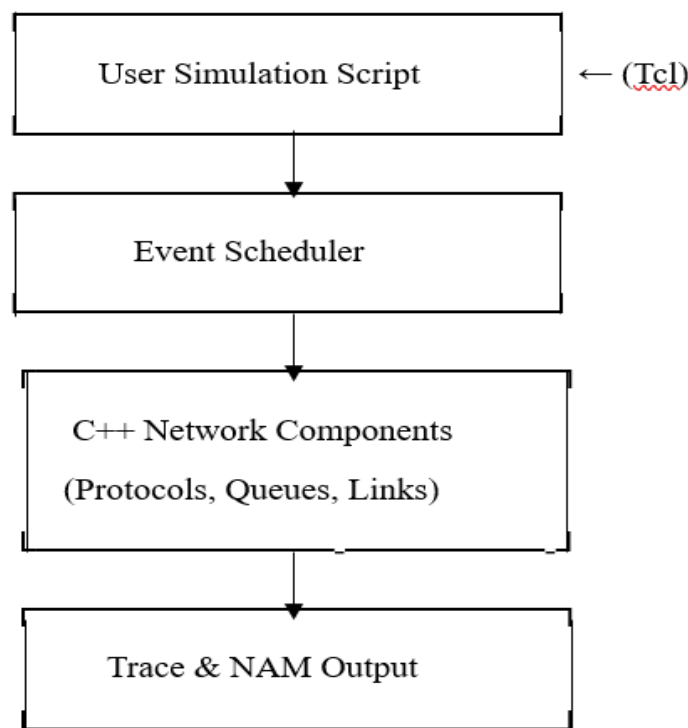
**Theory:**

NS2 (Network Simulator 2) is an open-source discrete event simulator primarily used for research and educational purposes in networking. It provides simulation support for both wired and wireless networks, and includes features to study routing, transport, and MAC layer protocols.

It allows users to model the behavior of TCP, UDP, IP, routing algorithms, and link-layer protocols. The simulation is defined using a Tcl (Tool Command Language) script, and results can be analyzed using NAM (Network Animator) or trace files.

**Main Components of NS2:**

1. **Tcl (Tool Command Language):**  
Used for writing simulation scripts that define nodes, links, agents, and traffic.
2. **C++ Backend:**  
Core network protocols and event handling are implemented in C++ for performance.
3. **NAM (Network Animator):**  
A graphical tool used to visualize network simulations.
4. **Trace Files:**  
Record detailed information of network events (packet send, drop, receive, delay, etc.).

**NS2 Architecture Overview:**

## Procedure:

### Step 1: Launch NS2

Step 2: Create a Simulation Script

Step 3: Run the Simulation

```
ns simple.tcl
```

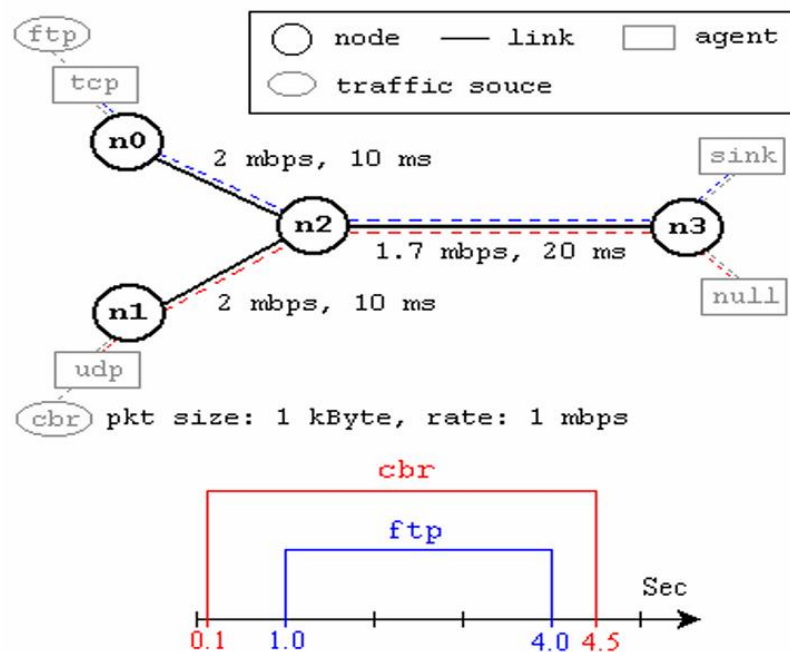
This will generate a file out.nam

### Step 4: View Simulation in NAM

To visualize:

```
nam out.nam
```

The network animator window will open, showing nodes, links, and packet movement.



### ns simple.tcl

```
#Create a simulator object
```

```
set ns [new Simulator]
```

```
#Define different colors for data flows (for NAM)
```

```
$ns color 1 Blue
```

```
$ns color 2 Red
```

```
#Open the NAM trace file
```

```
set nf [open out.nam w]
```

```
$ns namtrace-all $nf
```



#Define a 'finish' procedure

```
proc finish {} {  
    global ns nf  
    $ns flush-trace  
    #Close the NAM trace file  
    close $nf  
    #Execute NAM on the trace file  
    exec nam out.nam &  
    exit 0  
}
```

#Create four nodes

```
set n0 [$ns node]  
set n1 [$ns node]  
set n2 [$ns node]  
set n3 [$ns node]
```

#Create links between the nodes

```
$ns duplex-link $n0 $n2 2Mb 10ms DropTail  
$ns duplex-link $n1 $n2 2Mb 10ms DropTail  
$ns duplex-link $n2 $n3 1.7Mb 20ms DropTail
```

#Set Queue Size of link (n2-n3) to 10

```
$ns queue-limit $n2 $n3 10
```

#Give node position (for NAM)

```
$ns duplex-link-op $n0 $n2 orient right-down  
$ns duplex-link-op $n1 $n2 orient right-up  
$ns duplex-link-op $n2 $n3 orient right
```

#Monitor the queue for link (n2-n3). (for NAM)

```
$ns duplex-link-op $n2 $n3 queuePos 0.5
```

#Setup a TCP connection

set tcp [new Agent/TCP]

\$tcp set class\_ 2

\$ns attach-agent \$n0 \$tcp

set sink [new Agent/TCPSink]

\$ns attach-agent \$n3 \$sink

\$ns connect \$tcp \$sink

\$tcp set fid\_ 1

#Setup a FTP over TCP connection

set ftp [new Application/FTP]

\$ftp attach-agent \$tcp

\$ftp set type\_ FTP

#Setup a UDP connection

set udp [new Agent/UDP]

\$ns attach-agent \$n1 \$udp

set null [new Agent/Null]

\$ns attach-agent \$n3 \$null

\$ns connect \$udp \$null

\$udp set fid\_ 2

#Setup a CBR over UDP connection

set cbr [new Application/Traffic/CBR]

\$cbr attach-agent \$udp

\$cbr set type\_ CBR

\$cbr set packet\_size\_ 1000

\$cbr set rate\_ 1mb

\$cbr set random\_ false

#Schedule events for the CBR and FTP agents

\$ns at 0.1 "\$cbr start"

\$ns at 1.0 "\$ftp start"

\$ns at 4.0 "\$ftp stop"

\$ns at 4.5 "\$cbr stop"

#Detach tcp and sink agents (not really necessary)

\$ns at 4.5 "\$ns detach-agent \$n0 \$tcp ; \$ns detach-agent \$n3 \$sink"

#Call the finish procedure after 5 seconds of simulation time

\$ns at 5.0 "finish"

#Print CBR packet size and interval

puts "CBR packet size = [\$cbr set packet\_size\_]"

puts "CBR interval = [\$cbr set interval\_]"

#Run the simulation

\$ns run